

ARCHITECTURE_3_ SystemBandwidth

Earl Wong

Overview

- Previously, we discussed Latency in ARCHITECTURE_1_Latency.
- Previously, we discussed Real Time Pipeline in ARCHITECTURE_2_RealTimePipeline.
- Now, we discuss System Bandwidth, in ARCHITECTURE_3_SystemBandwidth.

Additional Comments on Latency

- In ARCHITECTURE_2_RealTimePipeline, we observed the use of BufferQueue.
- In CameraBufferQueue, 3 gralloc buffers were allocated.
- This increased memory usage by 3x.
- If only a single gralloc buffer is allocated, we incur 33 msec (30 fps) or 16.5 msec (60 fps) of additional latency.
- Since three gralloc buffers were allocated, latency increased to 99 msec (30 fps) and 49.5 msec (30 fps) respectively.

Additional Comments on Latency

- In WindowBufferQueue, we utilized a ping pong output buffer scheme.
- This increased memory usage by 2x.
- This increased latency by 66 msec (30 fps) and 33 msec (60 fps) respectively.
- Recall: In ARCHITECTURE_1_Latency, we assumed that a single gralloc buffer was allocated for the complete process.
- This produced 33 msec (30 fps) or 16.5 msec (60 fps) of latency.

Additional Comments on RealTimePipeline

- In ARCHITECTURE_2_RealTimePipeline, we walked through a real time pipeline that included GPU processing.
- In practice, GPU processing may not be needed / used.
- If true, only a single gralloc buffer would be needed (in theory) for write (camera) and read (display).
- In practice, 2 gralloc buffers are used, to prevent camera stalls.

Additional Comments on RealTimePipeline

- The non-GPU RealTimePipeline use case describes the minimal bandwidth requirement.
- Here, the Camera performs a write to DRAM, followed by a Display Controller read from the same DRAM.
- With this in mind, we let's discuss system bandwidth.

Overview

- Total Bandwidth Usage $\propto \sum$ (*all hardware reads + writes of frame buffer*)
- Total Bandwidth Usage =

$$\text{cost per pixel} * \text{number of pixels} * \text{frames per second} * \sum (\text{all hardware reads} + \text{writes of frame buffer})$$

Overview

- Number of Pixels (4K) = $3840 * 2160$; (1080) = $1920 * 1080$
- Frames per second = 60; 30

Cost Per Pixel:

- YUV, NV12, NV21 = 1.5 Bytes
- RGB888 = 3 Bytes
- RGBA8888 = 4 Bytes

Minimum System Bandwidth

Min System Requirements

- 4K, 60 fps, 1 write (NV12), 1 read (RGBA8888) =
- $1.5 \text{ Bytes} * (3840 * 2160) * 60 * 1 \text{ (read)} + 4 \text{ Bytes} * (3840 * 2160) * 60 + 1 \text{ (write)} = \sim 2.7 \text{ GBytes / sec}$

Cost of additional read and write = cost of copy

- $1.5 \text{ Bytes} * (3840 * 2160) * 60 * (1 \text{ read} + 1 \text{ write}) = \sim 1.5 \text{ GBytes / sec}$

Bandwidth Cost Growth

- In ARCHITECTURE_2_RealTimePipeline, we illustrated an example where additional copies were used.
- Specifically, CPU write and glTexImage read = copy.
- This action increased system bandwidth requirements by ~1.5 GBytes / sec.
- Depending on the number of additional read and write pairs, system bandwidth usage can rapidly increase beyond ~2.7 GBytes / sec.

Bandwidth Cost Growth

- In ARCHITECTURE_RealTimePipeline, we focused heavily on the data pipeline path, starting at the Camera and ending at the Display.
- We delved deeply into zero copy, illustrating how zero copy could occur, from passing a handle associated with each gralloc buffer.
- However, we omitted some necessary copies that occur in practice.
- We now mention these, since they result in bandwidth cost growth.

Bandwidth Cost Growth

- In practice, NV12 data needs to be converted to RGB data, for GLSL shader consumption.
- This requires an NV12 read and an RGB888 write.
- This means an additional ~2.2 GBytes / sec of bandwidth usage.

Bandwidth Cost Growth

- Suppose multiple shaders are needed, each operating at frame rate.
- i.e. Shader1 = temporal denoising, Shader2 = spatial denoising, Shader3 = sharpening and tone mapping.
- Now, not only have we incurred a 3x increase in latency, but also, a 3x increase in system bandwidth.
- i.e. $3 * 1.5 \text{ GBytes / sec.}$

Bandwidth Cost Growth

- Finally, consider the output, and composition of the processed frame with overlays.
- If SurfaceFlinger determines that hardware composer cannot perform the needed composition, the composition will need to occur in the GPU.
- If done in a serial fashion, $4 \text{ Bytes / sec} * (3840 * 2160) * 60 * 2 = \sim 4 \text{ GBytes / sec.}$

Worse Case

- Min Bandwidth = 2.7 GBytes / sec (1 write, 1 read)
- Worse Case Bandwidth = Min BW + GPU usage + 3 shader passes + GPU Composer = 2.7 GBytes / sec + 2.2 GBytes / sec + $(3 * 3840 * 2160 * 60) * 6 + 4$ GBytes / sec = 2.7 GBytes / sec + 2.2 GBytes / sec + 9 GBytes / sec + 4 GBytes / sec = 18 GBytes / sec
- Add multiple cameras = Wide + Normal + Tele = System Bandwidth Explosion => 3x
- Add encoder (read) and decoder (write) functionality...

Worse Case

Additional system processing will also compete for limited system bandwidth resources.

- Audio: Minimal
- ToF sensor: Could be significant, if sensor is full resolution.
- ML model deployment: Depends on use case.
- Typical image classification uses 224 x 224 subsampled input image and categorical output.
- But, consider the Apple iPhone face detector, which can detect faces >30 feet away ... input resolution needs to be >> greater than 224 x 224.
- But, consider high resolution ML based instance segmentation at frame rate ...

Relief 1

- In practice, SurfaceFlinger and the GPU composer may not need a separate serial step.
- This can save 4 GBytes / sec of additional bandwidth.

Relief 2

Frame buffer compression - can reduce bandwidth by $\frac{1}{2}$.

- Qualcomm - UBWC
- Mediatek ARM Mali - AFBC
- Samsung - SBWC

Relief 3

- Tile processing eliminates repeated DRAM data fetches.
- i.e. Using the correct shader eliminates unnecessary DRAM fetches.
- For example: Implementing a 31x31 non separable blur kernel in the GPU using a compute shader vs a fragment shader.
- The fragment shader is hardware optimized for local cache coherency.
- Hence, the fragment shader will require multiple data fetches at each pixel location.

Summary

- System bandwidth, latency, and real time pipelines are interrelated.
- At the heart of the matter, is the use of additional buffers and additional copy operations.
- The use of additional buffers may be unavoidable, to prevent pipeline stalls.
- The cost is increased system latency in the real time pipeline.

Summary

- The cost of the copy operation is extremely high.
- 1) Additional memory is used.
- 2) Additional system bandwidth is used.
- 3) Additional power is used.